1. Run BFS from v and let R be the set of reached vertices, $v \in R$ (the start is reached by convention). Then

$$v \text{ is on a cycle} \quad \Leftrightarrow \quad \exists (u \to v) \in E \text{ with } u \in R.$$

- $v \in R$, so a self-loop $(v, v) \in E$ is detected directly.
- Any other cycle through v consists of a nonempty path $v \xrightarrow{*} u$ plus the edge $u \to v$, i.e., exactly such a pair.

After the BFS, scan all edges once to check the condition. Both steps are $O(V + E)$.

2. Run DFS on G (over all vertices, restarting at unvisited ones) and let v be the vertex that finishes **last**. Then run BFS/DFS from v and check whether all vertices are reached: if yes, v is the desired vertex; if no, none exists. One full DFS plus one BFS from v: $O(V + E)$.

Correctness. Let $C(x)$ be the SCC of x; the condensation (SCCs as nodes) is a DAG.

(i) $C(v)$ is a **source** of the condensation. Otherwise some edge $x \to y$ enters it ($x \notin C(v)$, $y \in C(v)$).

- If x is discovered before v: at time $\text{disc}(x)$, v is white and x reaches v (via $x \to y \xrightarrow{*} v$), so by the white-path theorem v becomes a descendant of x and finishes before x — contradicting that v finishes last.
- If x is discovered after v: since v finishes last, $\text{finish}(x) < \text{finish}(v)$, so x is discovered while v is still active, i.e., x is a descendant of v and $v \xrightarrow{*} x$. But also $x \to y \xrightarrow{*} v$, so x and v reach each other, i.e., $x \in C(v)$ — contradiction.

(ii) If some vertex u reaches all others, then $C(u)$ reaches every node of the condensation DAG, so $C(u)$ is its unique source (any other source would have no incoming condensation edge, hence be unreachable from $C(u)$).

By (i) $C(v)$ is a source; by (ii) the source is unique and equals $C(u)$. Hence $v \in C(u)$ also reaches every vertex. □

3.

(a) 2-coloring by BFS. Start each unvisited component: color the source 1; when exploring an edge (u, w), give w the opposite color of u if w is newly discovered, and reject if w is already discovered with the **same** color as u. The graph is bipartite iff no edge ever joins two vertices of the same color. Each vertex and edge is processed once: $O(V + E)$.

(b) ($\Rightarrow$) In a bipartite graph a cycle must alternate between V_1 and V_2, so every cycle has even length; an odd cycle cannot exist.

($\leq$) Suppose G has no odd cycle. Run the algorithm of (a) from any source s in each component, and put $V_1 =$ even BFS levels, $V_2 =$ odd BFS levels. If some edge (u, w) joined two vertices on levels of the same parity, then since BFS levels of adjacent vertices differ by at most 1, u and w are on the **same** level i (and (u, w) is a non-tree edge). The tree paths $s \to u$ and $s \to w$ plus (u, w) form a closed walk of length $i + 1 + i = 2i + 1$, odd. A closed odd walk contains an odd cycle: take a shortest closed odd walk; it must be simple, since a repeated vertex splits it into two shorter closed walks, one of which is again odd. Contradiction. Hence no edge lies within one side, and G is bipartite. □

(c) 3. One odd cycle needs 3 colors on the cycle itself, and 3 always suffice: remove one edge (a, b) of the unique odd cycle; the remaining graph has no odd cycle, so it is 2-colorable by (b). Put (a, b) back and recolor a with a third color. This is consistent: a's neighbors other than b lie on the opposite side of the bipartition, hence all have the color a no longer uses, and b keeps its

old color — which is the **same** as a's old color, since the cycle minus (a, b) is an even-length path from a to b.

4. The minimum number of semesters equals the length (in edges) of the longest path in the prerequisite DAG, plus 1: any k-edge dependency path needs $k + 1$ semesters, and conversely taking every course v in semester $\text{sem}(v)$ is a feasible schedule since all prerequisites of v have smaller sem values. Compute by DFS with memoization over the given reverse-edge lists, using the recurrence (w ranges over the prerequisites of v):

$$\text{sem}(v) = \begin{cases} 1 & \text{if v has no prerequisites,} \\ 1 + \max\{\text{sem}(w) : w \to v\} & \text{otherwise,} \end{cases} \qquad \text{answer} = \max_{v \in V} \text{sem}(v).$$

Each course is computed once, each edge (reverse edge) used once: $O(n + m)$, linear. The answer is $\max_v \text{sem}(v)$. (If the graph has a cycle, the curriculum is infeasible.)

5.

(a) The precise statement (assuming the graph is connected, or restricted to each connected component of positive-degree vertices): **an undirected graph has an Eulerian tour iff the vertices of positive degree form a connected subgraph and every vertex has even degree.** Without the connectivity assumption the claim is false (two disjoint triangles have all degrees 2 but no tour).

($\Rightarrow$) In a tour, every time the walk enters a vertex it must also leave it by a different (unused) edge; entering and leaving pair up the incident edges, so every degree is even. The tour itself visits every edge, so all positive-degree vertices are in one connected piece.

($\leq$) Suppose all degrees are even and the positive-degree vertices are connected. Every vertex of positive degree lies on a cycle: walk from any vertex, always leaving by an unused edge; whenever the walk enters a vertex $u \neq s$ (the start), an odd number of u's edges have been used, so an unused edge is available to leave. With finitely many edges the walk must stop, and it can only get stuck at s, giving a cycle. To build a tour: start with any such cycle C; if C misses some edge, then by connectivity some vertex u on C is incident to a missed edge (the unused edges still have all even degrees, and no component of them is disjoint from C), find a cycle C' through u in the unused edges and splice C' into C at u. Repeat; each splice adds ≥ 1 edges, so it terminates with a closed walk using every edge exactly once: an Eulerian tour. $\square$

In the Konigsberg graph every vertex has odd degree — Big island has degree 5, and Northern bank, Small island, Southern bank each have degree 3 — so it has no Eulerian tour.

(c) A directed graph has an Eulerian tour iff (i) every vertex has in-degree = out-degree, and (ii) all vertices of nonzero degree belong to a single strongly connected component (equivalently: the underlying undirected graph on nonzero-degree vertices is connected, plus (i)).

Necessity: a tour enters each vertex as often as it leaves it, and the tour itself makes every nonzero-degree vertex mutually reachable. **Sufficiency:** same splicing argument as (a), using in = out to guarantee that whenever the walk enters a vertex it can leave it; strong connectivity supplies the analogue of (a)'s connectivity step — if edges remain, a directed path from the current tour to a vertex with an unused out-edge starts with an unused edge whose tail lies on the tour, so a new cycle can be spliced in.

6.

(a) The satisfying assignments are exactly: $x_1 = \text{true}, x_3 = \text{false}, x_4 = \text{true}$, with x_2 free. So there are 2 in total:

$$(x_1, x_2, x_3, x_4) = (T, F, F, T) \quad \text{and} \quad (T, T, F, T).$$

(If $x_1 = F$: clause $(x_1 \vee x_2)$ forces $x_2 = T$, while $(x_1 \vee \overline{x_2})$ forces $x_2 = F$ — contradiction; if $x_1 = T$: $(\overline{x_1} \vee \overline{x_3})$ forces $x_3 = F$, $(\overline{x_1} \vee x_4)$ forces $x_4 = T$, and x_2 is unconstrained.)

(b) $(x_1 \vee x_2) \wedge (x_1 \vee \overline{x_2}) \wedge (\overline{x_1} \vee x_2) \wedge (\overline{x_1} \vee \overline{x_2}) \wedge (x_3 \vee x_4)$. The first four clauses are satisfiable by no value of x_1, x_2 (they assert $x_1 = T$ and $x_1 = F$), so the instance is unsatisfiable; x_3, x_4 appear in the last clause.

(c) Implication graph for the instance in the statement, 8 nodes; each clause $(\alpha \vee \beta)$ gives edges $\overline{\alpha} \to \beta$ and $\overline{\beta} \to \alpha$:

$$\begin{aligned}
(x_1 \vee \overline{x_2}): &\quad \overline{x_1} \to \overline{x_2}, \quad x_2 \to x_1 \\
(\overline{x_1} \vee \overline{x_3}): &\quad x_1 \to \overline{x_3}, \quad x_3 \to \overline{x_1} \\
(x_1 \vee x_2): &\quad \overline{x_1} \to x_2, \quad \overline{x_2} \to x_1 \\
(\overline{x_3} \vee x_4): &\quad x_3 \to x_4, \quad \overline{x_4} \to \overline{x_3} \\
(\overline{x_1} \vee x_4): &\quad x_1 \to x_4, \quad \overline{x_4} \to \overline{x_1}
\end{aligned}$$

For the instance in (b):

$$\begin{aligned}
(x_1 \vee x_2): &\quad \overline{x_1} \to x_2, \quad \overline{x_2} \to x_1 \\
(x_1 \vee \overline{x_2}): &\quad \overline{x_1} \to \overline{x_2}, \quad x_2 \to x_1 \\
(\overline{x_1} \vee x_2): &\quad x_1 \to x_2, \quad \overline{x_2} \to \overline{x_1} \\
(\overline{x_1} \vee \overline{x_2}): &\quad x_1 \to \overline{x_2}, \quad x_2 \to \overline{x_1} \\
(x_3 \vee x_4): &\quad \overline{x_3} \to x_4, \quad \overline{x_4} \to x_3
\end{aligned}$$

Here $x_1 \to x_2 \to x_1$ and $x_1 \to \overline{x_2} \to x_1$ put $x_1, x_2, \overline{x_2}$ in one component, and $\overline{x_1} \to x_2$ (clause $x_1 \vee x_2$) together with $x_2 \to \overline{x_1}$ (clause $\overline{x_1} \vee \overline{x_2}$) brings in $\overline{x_1}$: so $x_1, \overline{x_1}, x_2, \overline{x_2}$ all lie in one strongly connected component.

(d) If x and $\overline{x}$ are in the same SCC, then G_I contains paths $x \Rightarrow \overline{x}$ and $\overline{x} \Rightarrow x$, i.e., the implications $x \Rightarrow \overline{x}$ and $\overline{x} \Rightarrow x$. Every edge of G_I is a valid implication of I, and implications transitive: any satisfying assignment makes every literal along a path true if its source is true. $x \Rightarrow \overline{x}$ says: if $x = T$ then $\overline{x} = T$, impossible; $\overline{x} \Rightarrow x$ says: if $x = F$ then $x = T$, impossible. Either truth value of x violates an implication, so no satisfying assignment exists.

(e) Assume no SCC contains both a literal and its negation. Process SCCs from sinks backwards (equivalently, in reverse topological order of the condensation): repeatedly pick a sink SCC S of the remaining graph, assign **true** to all literals in S, **false** to their negations, and delete S together with $\overline{S}$.

Invariant: every literal assigned true so far only implies literals that are also true at the end. When S is picked as a sink, all outgoing edges of S lead to vertices inside S itself or to already-deleted components that were assigned **true**; so everything reachable from a literal of S is true. (An edge from S into an earlier-deleted **false** component $\overline{T}$ is impossible: by skew-symmetry it would give an edge $T \to \overline{S}$, and when T was picked, $\overline{S}$ was still present — pairs are deleted together — so T had an outgoing edge, contradicting that T was a sink then.) Negations: $\left\{\overline{\ell} : \ell \in S\right\}$ form an SCC $\overline{S} \neq S$ (G_I is closed under negation, and an edge $A \to B$ between SCCs exists iff $\overline{B} \to \overline{A}$ does). At each step delete $S \cup \overline{S}$: assign true to S, false to $\overline{S}$. When S is picked, $\overline{S}$ is still undeleted: if some $\overline{\ell}$ with $\ell \in S$ had been deleted in an earlier step, then in that same

step its component $Tni\overline{\ell}$ and the complementary component $\overline{T}ni\ell$ were both deleted — so ℓ was already deleted, contradicting $\ell \in S$. Each variable thus receives exactly one value, consistently.

All clauses satisfied: suppose clause $(\alpha \vee \beta)$ ended with both literals false, i.e., $\overline{\alpha}$ and $\overline{\beta}$ true. The edge $\overline{\alpha} \to \beta$ is in G_I, so β is reachable from a true literal, hence true by the invariant — contradiction. $\square$

(f) Construct G_I ($2n$ nodes, $2m$ edges) in $O(n+m)$; compute SCCs by a linear-time algorithm (Tarjan / Kosaraju, $O(V+E)$); by (d)–(e), I is satisfiable iff no SCC contains both x and $\overline{x}$, and the procedure of (e) produces an assignment. Total time $O(n+m)$: linear. $\square$